

Project Description

Students must select one of the following systems and implement it using Java classes and inheritance.

Each project must include:

- At least one base (parent) class
- At least two child (sub)classes
- Use of:
 - attributes
 - methods
 - constructors
 - inheritance (extends)
 - object creation

1. Select a real-world object or system :

Hospital Management System Concepts: Aggregation, array of objects Description: Classes: Doctor, Patient, Appointment, Department. <i>Challenge:</i> Link patients to doctors based on department and availability.	Game Character Battle System Concepts: Inheritance, encapsulation, interfaces Description: Abstract class: Character Subclasses: Warrior, Wizard, Archer. Each class defines unique attack power and special move. <i>Challenge:</i> Simulate a fight round between objects and display winner.
Payroll System with Polymorphism Concepts: Polymorphism, inheritance, abstract classes Description: Base class: Employee	Commerce Order Processing System Concepts: Aggregation, composition, array of objects Description: Classes: Customer, Order, Product, Payment.

Subclasses: FullTimeEmployee, PartTimeEmployee, Contractor. Each has calculateSalary() method with different logic. <i>Challenge:</i> Use array or ArrayList<Employee> to process all salaries polymorphically	Each class interacts to simulate a full purchase process. <i>Challenge:</i> Add validation for inventory and discounts for VIP customers
Autonomous Car Simulator Concepts: Encapsulation, inheritance, decision-making Description: Create classes: • Car (base class) • Sensor (detect obstacles) • AIController (make decisions like stop/turn) <i>Challenge:</i> Implement methods where AIController reads Sensor data and decides movement.	Smart Home Automation System Concepts: Inheritance, polymorphism, interfaces Description: Build a system with abstract Device class and subclasses like Light, Fan, and Thermostat. Include an interface Controllable with turnOn() and turnOff() methods. <i>Challenge:</i> Use polymorphism to handle devices in a single list and control them dynamically.
University Course Registration System Concepts: Composition, encapsulation, constructor chaining Description: Classes: Student, Course, Registration. Methods to enroll, drop, and display registered courses. <i>Challenge:</i> Validate course capacity and prerequisites.	Banking System with Inheritance Concepts: Inheritance, overriding, encapsulation Description: Base class: Account Subclasses: SavingsAccount, CurrentAccount Each subclass overrides calculateInterest() and withdraw() rules. <i>Challenge:</i> Implement validation logic and monthly interest computation.

2. General Coding Requirements

Each project must contain:

- At least 3 attributes per class
- At least 2 methods per class
- A constructor in each class
- At least one overridden method
- A main class to test the system

3. Video Structure

Section	Duration	Description
Introduction	1 min	Student introduces self, project title, and topic overview
Code Explanation	3–4 mins	Walkthrough of attributes, methods, constructor, and output
Demonstration	2 mins	Run the code and show the program in action
Peer Q&A	2–3 mins	5 classmates ask short questions about the project
Reflection	1 min	Presenter explains what they learned or would improve

Rules for the Q&A Segment

- The 5 classmates **must appear** in the recording video.
- Each classmate asks **1 short, meaningful question** such as:
 - “Why did you use a constructor instead of setter methods?”
 - “Can you explain what happens if you remove this method?”
 - “Why is this variable private?”
 - “What happens if we create two objects with different values?”
- Presenter must answer immediately — no reading or editing.
- Questions must relate directly to the code or logic.

4. Submission method

Students must upload their presentation video to **YouTube** and upload their **Java source files together with the video link** to their **GitHub repository**.

The **GitHub repository link** must be submitted on **Spectrum** before the deadline.

Criteria	Description	Weight
Understanding of Class Concept	Correct explanation of class, object, attributes, and methods	2.5
Code Implementation	Working example, syntax correctness, and logic	2.5
Presentation Skills	Clarity, organization, and confidence	2
Application & Creativity	Real-world relevance or innovative example	2
Engagement & Delivery	Visuals, explanation style, and reflection	1